LangChain RAG Pipeline

Technical Reference & Core Functions Guide

> Retrieval-Augmented Generation (RAG) applications require a strict chronological pipeline. Data must flow sequentially from loading and chunking to embedding, storage, and retrieval before the Large Language Model (LLM) can generate a final response.

1. Core Pipeline Steps

Step 1: Document Loading

Ingest raw data into standardized LangChain Document objects.

- **Modules:** PyPDFLoader, TextLoader, WebBaseLoader
- **Execution:** `docs = loader.load()`

Step 2: Text Splitting (Chunking)

Slice large documents into smaller overlapping chunks to fit within LLM context windows.

- **Modules:** RecursiveCharacterTextSplitter
- **Key Config:** chunk_size (e.g., 1000 chars), chunk_overlap (e.g., 200 chars).
- **Execution:** `chunks = text_splitter.split_documents(docs)`

Step 3: Embedding & Storage

Convert text chunks into vector representations and store them in a vector database for rapid similarity search.

- **Modules:** OpenAIEmbeddings, HuggingFaceEmbeddings, Chroma (VectorStore)
- **Execution:** `vectorstore = Chroma.from_documents(chunks, embeddings)`

Step 4: Retrieval

Create an interface that fetches the most semantically relevant chunks for a given user query.

- **Execution:** `retriever = vectorstore.as_retriever(search_kwargs={"k": 3})`
- **Details:** The k parameter determines how many context chunks are retrieved.

Step 5: Chain Assembly & Generation

Connect the prompt, the LLM, and the retriever to form the final question-answering chain.

- **Modules:** create_stuff_documents_chain, create_retrieval_chain
- **Execution:** Combine the document chain with the retriever, then call `chain.invoke({"input": query})`.

2. Quick Reference Table

Concept	Primary LangChain Function	Purpose
Loader	`PyPDFLoader(path).load()`	Reads raw files into memory.
Splitter	`RecursiveCharacterTextSplitter(...)`	Chunks text to fit token limits.
Embeddings	`OpenAIEmbeddings()`	The AI model used to vectorize text.
Vector Store	`Chroma.from_documents(...)`	Stores and indexes the vectors.
Retriever	`vectorstore.as_retriever()`	Fetches similar chunks based on queries.

3. Complete Example Code

Integration Note: This example demonstrates how all 5 steps wire together into a single, functional script. You can easily swap TextLoader for PyPDFLoader depending on your source data.

```
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import Chroma
from langchain.chains import create_retrieval_chain
from langchain.chains.combine_documents import create_stuff_documents_chain
from langchain_core.prompts import ChatPromptTemplate

# 1. Load
loader = TextLoader("data.txt")
docs = loader.load()

# 2. Split
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks = splitter.split_documents(docs)

# 3. Embed & Store
embeddings = OpenAIEmbeddings()
vectorstore = Chroma.from_documents(chunks, embeddings)

# 4. Retrieve
retriever = vectorstore.as_retriever(search_kwargs={"k": 2})

# 5. Generate
llm = ChatOpenAI(model="gpt-3.5-turbo")
prompt = ChatPromptTemplate.from_template(
    "Answer the question based only on context:\n\n{context}\n\nQuestion: {input}"
)
doc_chain = create_stuff_documents_chain(llm, prompt)
rag_chain = create_retrieval_chain(retriever, doc_chain)

# Run
response = rag_chain.invoke({"input": "What is the main topic?"})
print(response["answer"])
```

Architecture Diagram